

UN AMM $x \cdot y = k$, UN ORÁCULO REAL Y LA PÉRDIDA IMPERMANENTE

Completar y medir un mini-AMM, consumir el price feed ETH/USD de Chainlink en Sepolia con sus defensas, y calcular la pérdida impermanente en una hoja que se autocomprueba.

MODALIDAD	REQUISITO	DURACIÓN	META
En parejas dentro del equipo	laboratorios-evm, keystore y hoja	65 min en clase + Sepolia en casa	21 pruebas + 1 contrato verificado + 4 ✓

CÓMO LEER ESTA GUÍA

Tres partes independientes: si una se atasca, avanza con la otra. Cada paso dice qué escribir, qué deberías ver y cómo saber que funcionó. Todos los números esperados de esta guía salieron de correr el código del laboratorio; si tu resultado es otro, el error está en tu código, no en la guía. Los problemas comunes están en la sección 12.

01 Qué vas a construir, y por qué

LEER PRIMERO

Objetivo. Entender las primitivas DeFi como quien las evalúa: medir el precio, el slippage y la pérdida impermanente de un AMM, y leer un precio externo sin caer en las dos trampas clásicas de los oráculos (precio rancio y precio manipulable). **Resultados de aprendizaje:** RA2 (justificar una arquitectura), RA5 (riesgos y defensas) y RA7 (implicaciones económicas).

PARTE	QUÉ HACES	ARCHIVOS
1 · Mini-AMM	Completas <code>cotizar</code> , <code>intercambiar</code> y <code>quitarLiquidez</code> ; mides slippage y pérdida impermanente en cadena.	<code>andamiaje/s14/PoolXYK.sol</code> , <code>test/s14/PoolXYK.test.js</code> , <code>scripts/s14/pool-en-numeros.js</code>
2 · Oráculo	Completas las defensas del consumidor y lo despliegas en Sepolia contra el feed real de Chainlink.	<code>andamiaje/s14/OraculoConsumidor.sol</code> , <code>test/s14/Oraculo.test.js</code> , <code>scripts/s14/leer-chainlink.js</code>
3 · Hoja de cálculo	Tres escenarios de precio con fórmulas; la plantilla dice si coinciden.	<code>guias/s14-plantilla-perdida-impermanente.xlsx</code> (y <code>.csv</code>)

La interfaz de Uniswap admite Sepolia en «modo testnet» (verificado el 17 de septiembre de 2026 en su centro de soporte). Pero los pools de prueba tienen poca liquidez y precios sin relación con ningún mercado, y cada paso exige tokens y ETH de prueba. Medir slippage allí mide el desorden del pool, no el mecanismo. Por eso la parte 1 usa `PoolXYK`: la misma idea de Uniswap v2 en 150 líneas que puedes leer, probar y medir. Uniswap queda como extensión opcional (sección 11).

02 Qué se entrega

EVIDENCIA VERIFICABLE

1. **Mini-AMM:** captura de `15 passing` en `test/s14/PoolXYK.test.js` con tu código, y la salida de `pool-en-numeros.js`.
2. **Oráculo:** captura de `6 passing` en `test/s14/Oraculo.test.js`, dirección de tu `OraculoConsumidor` **verificado** en Sepolia y la salida de `leer-chainlink.js` con `CONSUMIDOR`.
3. **Hoja:** tu archivo con las cuatro filas en «✓ coincide», hechas con fórmulas.
4. **Análisis:** un párrafo sobre un ataque real visto en clase (oráculo o flash loan): qué falló y qué defensa de este laboratorio lo habría evitado.
5. Las respuestas de reflexión (sección 13).

Un documento por pareja, por el canal del curso. Sin evidencia en cadena (el consumidor verificado en Sepolia), el laboratorio no se califica.

03 Preparación desde cero

10 MINUTOS

```
node --version          # v22.x
cd laboratorios-evm
git pull                # trae contracts/s14, test/s14, scripts/s14, andamiaje/s14 y la
                        # plantilla
npm install              # solo si es la primera vez en este computador
npx hardhat --version   # 3.16.0
npx hardhat keystore list # SEPOLIA_RPC_URL, SEPOLIA_PRIVATE_KEY, ETHERSCAN_API_KEY
```

Cómo sé que funcionó: existe `guias/s14-plantilla-perdida-impermanente.xlsx` y `npx hardhat test test/s14/PoolXYK.test.js` compila (cuántas pruebas pasan lo ves en el paso 1). Para la hoja necesitas Excel, LibreOffice Calc o Google Sheets; cualquiera sirve.

REGLAS QUE NO ADMITEN EXCEPCIÓN

Solo red local y Sepolia: **nunca** dinero real, y nada de este laboratorio es una recomendación de inversión. La clave privada es la de la billetera del curso y vive **solo** en el keystore, igual que la URL del RPC. La dirección de un contrato no es un secreto: puede ir en una variable de entorno. Los tokens TETH y TUSD de la parte 1 son de juguete: cualquiera puede acuñarlos.

04 Parte 1 · Paso 1: el pool en rojo

5 MINUTOS

```
copy andamiaje\s14\PoolXYK.sol contracts\s14\      (Windows)
cp andamiaje/s14/PoolXYK.sol contracts/s14/        (macOS / Linux)
npx hardhat test test/s14/PoolXYK.test.js
```

Salida esperada: compila (con avisos de «pure» del andamiaje) y termina en `3 passing` y `12 failing`. Las tres verdes son las de `agregarLiquidez`, que ya viene hecha: léela antes de seguir, porque te muestra cómo se usan `reservaA`, `reservaB`, `totalSupply()` y `safeTransferFrom`.

El pool de las pruebas es el de la clase: **10 TETH y 20 000 TUSD**, precio 2 000, comisión 0,30 % (`COMISION_BPS = 30`, `BPS = 10 000`).

05 Parte 1 · Paso 2: TODO 1, cotizar

LA FÓRMULA

Quien mete `montoEntrada` de un token saca del otro la cantidad que mantiene el producto de las reservas. Con la comisión descontada de la entrada:

```
entradaNeta = montoEntrada * (1 - 0,003)
(rEntrada + entradaNeta) * (rSalida - salida) = rEntrada * rSalida
⇒ salida = rSalida * entradaNeta ÷ (rEntrada + entradaNeta)
```

Cuánto sale, en enteros

Qué escribir (debajo de la línea que ya obtiene `rEntrada` y `rSalida`):

```
· if (montoEntrada == 0) revert MontoCero();  
· if (rEntrada == 0 || rSalida == 0) revert SinLiquidez();  
· uint256 entradaNeta = montoEntrada * (BPS - COMISION_BPS); ← ya multiplicada por 10 000  
· return (rSalida * entradaNeta) / (rEntrada * BPS + entradaNeta);
```

Por qué así: Solidity no tiene decimales. Para aplicar 0,997 se multiplica por 9 970 y se multiplica también el denominador por 10 000. Y siempre se multiplica **antes** de dividir: la división entera trunca.

Comprobación a mano: 2 000 TUSD → $\text{entradaNeta} = 2\,000 \times 9\,970 = 19\,940\,000$ (en unidades de 10^{18}); $\text{salida} = 10 \times 19\,940\,000 \div (20\,000 \times 10\,000 + 19\,940\,000) = 0,906610893880149131$ TETH. La prueba «el ejemplo de la lámina» exige ese valor exacto: 906610893880149131 unidades mínimas.

2

FUNCIÓN · intercambiar(tokenEntrada, montoEntrada, minimoSalida)

Checks → effects → interactions (Sesión 9)

Checks: salida = cotizar(tokenEntrada, montoEntrada); y if (salida < minimoSalida) revert SlippageExcedido(minimoSalida, salida);

Effects: bool entraA = tokenEntrada == address(tokenA); Si entra A: reservaA += montoEntrada; reservaB -= salida; . Si entra B, al revés.

Interactions: IERC20(tokenEntrada).safeTransferFrom(msg.sender, address(this), montoEntrada); y (entraA ? tokenB : tokenA).safeTransfer(msg.sender, salida); . Al final, emit Intercambio(msg.sender, tokenEntrada, montoEntrada, salida);

Lo que vigilan las pruebas: que entregues exactamente lo cotizado, que k nunca baje (sube un poco por la comisión) y que un mínimo no alcanzado revierta todo (★ slippage).

3

FUNCIÓN · quitarLiquidez(participaciones)

La parte proporcional de AMBAS reservas

- if (participaciones == 0) revert MontoCero();
- uint256 total = totalSupply(); — **antes de quemar:** si quemas primero, el total cambia y el cálculo sale mal.
- montoA = (participaciones * reservaA) / total; y lo mismo para B.
- _burn(msg.sender, participaciones); — **revierte sola con ERC20InsufficientBalance** si no las tiene.
- Restar de las reservas, safeTransfer de los dos tokens y emit LiquidezRetirada(msg.sender, montoA, montoB, participaciones);

Cómo sé que terminé: `npx hardhat test test/s14/PoolXYK.test.js` muestra 15 passing, incluidas las tres con ★:

- ✓ ★ protección de slippage: si sale menos del mínimo aceptado, se revierte todo
- ✓ ★ pérdida impermanente en cadena: si el precio se multiplica por ~4, el LP pierde ≈ 20 % frente a conservar
- ✓ ★ una sola compra grande multiplica el precio spot dentro de la misma transacción

15 passing

Con `--coverage`, la solución de referencia da 100 % de líneas en `PoolXYK.sol`.

```
npx hardhat run scripts/s14/pool-en-numeros.js
```

Salida real en la red local (el gas puede variar unas unidades):

```

POOL INICIAL: 10 TETH + 20 000 TUSD · comisión 0,30 %
inicio          TETH      10,0000 · TUSD      20.000,00 · k      200.000,00 · precio      2.000,00

1 · COTIZAR (lectura, sin gas)
TUSD que entran  TETH que salen  precio promedio  peor que el spot
    100          0,049603        2.016,02         0,80 %
    2.000        0,906611        2.206,02         10,30 %
    10.000       3,326660        3.006,02         50,30 %

2 · COMPRAR con 2 000 TUSD, tolerancia de slippage del 1 %
recibió 0,906611 TETH (mínimo aceptado 0,897545) · gas 69751
después de comprar  TETH      9,0934 · TUSD      22.000,00 · k      200.054,56 · precio      2.419,34

3 · PÉRDIDA IMPERMANENTE, MEDIDA EN CADENA
el arbitrajista mete 20 000 TUSD; nuevo precio 7.988,00 (r = 3,9940)
el LP retiraría      5,0075 TETH + 40.000,00 TUSD = 80.000,00 TUSD
si hubiera conservado 10 TETH + 20 000 TUSD      = 99.880,00 TUSD
pérdida impermanente medida 19,90 % · fórmula  $2\sqrt{r/(1+r)}$  19,96 %

```

Cómo sé que funcionó: los números coinciden con los de arriba. Si la parte 1 da otros valores, tu `cotizar` tiene un error: vuelve a la comprobación a mano del paso 2.

08 Parte 2 · El consumidor de oráculo

20 MINUTOS

```

copy andamiaje\s14\OraculoConsumidor.sol contracts\s14\      (cp en macOS / Linux)
npx hardhat test test\s14\Oraculo.test.js                  # al empezar: 0 passing · 6 failing

```

4

FUNCIÓN · `precioEth(maxAntiguedad)` · TODO 1 A 3

Leer un precio sin creérselo todo

TODO 1 · positivo: `if (answer <= 0) revert PrecioInvalido();` — un feed puede devolver 0 o negativo si algo falla.

TODO 2 · no rancio: `uint256 antiguedad = block.timestamp - updatedAt;` y `if (antiguedad > maxAntiguedad) revert PrecioViejo(antiguedad);` — un oráculo que dejó de actualizarse es tan peligroso como uno manipulado: en una caída, seguiría diciendo el precio de antes.

TODO 3 · normalizar: `uint8 d = feed.decimals();` y `return uint256(answer) * (10 ** (18 - d));` — Chainlink ETH/USD usa 8 decimales; los tokens, 18.

Cómo lo compruebas: 6 passing. Mira las dos con ★: «rechaza un precio rancio» y «demuestra la manipulación: quien controla el oráculo controla el precio».

El feed real, verificado

DATO	VALOR	DÓNDE SE VERIFICÓ
Dirección ETH/USD en Sepolia	0x694AA1769357215DE4FAC081bf1f309aDC325306	docs.chain.link → Price Feed Contract Addresses → Ethereum → Sepolia (17-sep-2026)
Decimales · desviación · heartbeat	8 · 1 % · 3 600 s	La misma página, «More details».
description()	"ETH / USD"	Lectura directa del contrato en Sepolia.

Antes de usarla en otro semestre, vuelve a comprobarla en `docs.chain.link` : las direcciones de redes de prueba pueden cambiar.

09 Parte 2 · Desplegar en Sepolia y leer

LA EVIDENCIA EN CADENA

9.1 · Leer el feed directo (solo lectura, no gasta ETH):

```
npx hardhat run scripts/s14/leer-chainlink.js --network sepolia
```

Salida real del 17-sep-2026 (la hora está en UTC; tus números serán otros):

```
Red: Sepolia · feed real de Chainlink 0x694AA1769357215DE4FAC081bf1f309aDC325306
roundId      : 18446744073709587749
answer (crudo): 244571660075 (8 decimales)
precio ETH/USD: 2445.71660075
actualizado   : 2026-09-18T00:04:00.000Z · hace 2952 s
¿rancio?      : no (heartbeat 3600 s)
```

9.2 · Desplegar tu consumidor, verificado:

```
npx hardhat ignition deploy ignition/modules/s14-OraculoConsumidor.js --network sepolia --verify
```

El módulo trae la dirección del feed como parámetro `feed` por defecto. Salida esperada:

`OraculoConsumidorModulo#OraculoConsumidor - 0x...` y el enlace de verificación.

9.3 · Leer a través de tu contrato:

```
$env:CONSUMIDOR="0x...tu consumidor..." (PowerShell)
export CONSUMIDOR=0x...tu consumidor... (macOS / Linux)
npx hardhat run scripts/s14/leer-chainlink.js --network sepolia
```

Al final aparecen dos líneas más: `Su OraculoConsumidor 0x...` y `precioEth(7200) : 2445.71660075` (o el precio de ese momento), ahora con 18 decimales. **Cómo sé que terminé:** el precio por tu contrato coincide con el

crudo, y en Etherscan tu contrato tiene la marca de verificado.

10 Parte 3 · La hoja de pérdida impermanente

15 MINUTOS

Abre `guias/s14-plantilla-perdida-impermanente.xlsx`. Un LP depositó **1 ETH + 2 000 USDC con ETH a 2 000** (celdas B3 a B5). Llena las celdas **amarillas** de las filas 9 a 12 con **fórmulas**, no con números copiados:

COLUMNA	QUÉ ES	FÓRMULA (FILA 9; LUEGO SE ARRASTRA)
C · r	cuántas veces cambió el precio	<code>=B9/\$B\$5</code>
D · ETH en el pool	$x_0 \div \sqrt{r}$	<code>=B\$3/SQRT(C9)</code> · en Excel en español: <code>=B\$3/RAIZ(C9)</code>
E · USDC en el pool	$y_0 \times \sqrt{r}$	<code>=B\$4*SQRT(C9)</code>
F · valor del pool	ETH × precio final + USDC	<code>=D9*B9+E9</code>
G · valor si conserva	$x_0 \times \text{precio final} + y_0$	<code>=B\$3*B9+B\$4</code>
H · pérdida	$1 - \text{pool} \div \text{conservar}$	<code>=1-F9/G9</code>

Valores esperados (también están en la hoja, debajo, y en el `.csv`):

ESCENARIO	r	ETH	USDC	POOL	CONSERVAR	PÉRDIDA
A · ETH cae a 1 000	0,5	1,4142	1 414,21	2 828,43	3 000	5,72 %
B · ETH sube a 4 000	2	0,7071	2 828,43	5 656,85	6 000	5,72 %
C · ETH sube a 8 000	4	0,5000	4 000,00	8 000,00	10 000	20,00 %

Cómo sé que funcionó: la columna I dice «✓ coincide» en A, B y C. En la fila D escribe un precio final propio en B12 (por ejemplo 20 000, $r = 10 \rightarrow 42,50\%$): la columna I la contrasta con la fórmula cerrada $1 - 2\sqrt{r} \div (1 + r)$. Los mismos números salen de `node scripts/s14/perdida-impermanente.js 10`.

SEPARADORES SEGÚN EL PROGRAMA

En Excel o LibreOffice configurados en español, los argumentos se separan con `;` y la raíz se llama `RAIZ`; en inglés y en Google Sheets, `SQRT`. Las fórmulas que ya trae la plantilla se traducen solas al abrirla. Si todo marca «X revisar» aunque los valores parezcan iguales, escribiste números redondeados en vez de fórmulas.

11 Para ir más allá (opcional)

- **Uniswap en Sepolia.** En `app.uniswap.org`, con la billetera del curso, activa el modo testnet en la configuración y elige Sepolia. Haz un intercambio pequeño y anota el precio cotizado, el mínimo a recibir y el

obtenido. Compara con la tabla del paso 4 y explica por qué un pool de prueba se comporta distinto. Lee cada campo antes de firmar: una aprobación ilimitada de tokens también es una firma.

- **Circuit breaker.** Añade a tu copia de `OraculoConsumidor` una comprobación que rechace el precio si cambió más de un X % respecto a la última lectura guardada, y una prueba que lo demuestre con `OraculoFalso.fijarPrecio`.
- **Sin comisión.** En una copia del pool pon `COMISION_BPS = 0` y vuelve a correr el paso 4: la pérdida medida debe igualar la fórmula.

12 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

SÍNTOMA	CAUSA Y SOLUCIÓN
«el ejemplo de la lámina» falla por poco	Dividiste antes de multiplicar, o aplicaste la comisión a la salida. Revisa el orden de la fórmula del paso 2.
cotizar devuelve 0	Olvidaste multiplicar <code>rEntrada</code> por <code>BPS</code> en el denominador: el numerador está 10 000 veces «inflado» y el denominador no.
«k nunca baja» falla	Sumaste <code>entradaNeta</code> (sin comisión) a la reserva en vez de <code>montoEntrada</code> . La comisión debe QUEDARSE en el pool.
«retirar todas las participaciones» da montos raros	Leíste <code>totalSupply()</code> después de <code>_burn</code> . Léelo antes.
<code>ERC20InsufficientAllowance</code> en un script propio	Falta <code>approve</code> del usuario al pool. En las pruebas ya está hecho.
<code>PrecioViejo</code> en Sepolia	<code>maxAntiguedad</code> menor que el heartbeat de 3 600 s. El script usa 7 200.
<code>precioEth</code> da un número 10^{10} veces mayor	Multiplicaste por <code>10 ** 18</code> en vez de <code>10 ** (18 - d)</code> .
En Sepolia «insufficient funds»	Sin ETH de prueba en la billetera del curso: pídelo al docente.
La verificación en Etherscan falla	Falta <code>ETHERSCAN_API_KEY</code> o Etherscan aún no ve el contrato. Espera y corre <code>npx hardhat ignition verify chain-11155111</code> .
La hoja marca X con valores iguales a la vista	Números tecleados y redondeados en vez de fórmulas, o la celda del precio final vacía.

13 Preguntas de reflexión

CON TUS PROPIOS VALORES

1. **k.** Copia de tu salida del paso 4 el valor de `k` antes y después de la compra. ¿Cuánto creció, y cuánto debería crecer según la comisión cobrada? (Pista: 0,30 % de 2 000 TUSD, al precio del momento.)
2. **Medida frente a fórmula.** Con tus números, ¿cuánto vale la diferencia entre la pérdida medida y la de la fórmula, en puntos porcentuales? ¿A quién se le quedó esa diferencia?
3. **Simetría.** Con tu hoja, explica por qué A y B dan el mismo porcentaje aunque en A el ETH cae y en B sube. ¿El LP termina con más ETH o con más USDC en cada caso?

4. **Comisiones contra pérdida.** Para el escenario C, ¿qué volumen diario de intercambios necesita un pool de 40 000 USDC para que el LP recupere el 20 % en un año con comisión de 0,30 %? Muestra el cálculo.
5. **El feed.** En tu corrida de Sepolia, copia `updatedAt` y la antigüedad. ¿Qué habría pasado con `precioEth(600)` en ese momento? ¿Qué valor pondrías en un contrato real que vende boletas, y por qué?
6. **Manipulación.** La prueba ★ del precio spot mete 100 000 TUSD. Calcula a mano el precio spot después (reservas nuevas) y explica en tres líneas por qué ningún contrato debería usar `precioSpotA()` como oráculo.

14 Lista de chequeo de entrega

✓	ÍTEM
☐	15 <code>passing</code> en <code>test/s14/PoolXYK.test.js</code> con tu código, y salida de <code>pool-en-numeros.js</code> .
☐	6 <code>passing</code> en <code>test/s14/Oraculo.test.js</code> con tu código.
☐	<code>OraculoConsumidor</code> verificado en Sepolia: dirección y enlace.
☐	Salida de <code>leer-chainlink.js --network sepolia</code> con <code>CONSUMIDOR</code> definido.
☐	Hoja con las filas A, B, C y D en «✓ coincide», hecha con fórmulas.
☐	Párrafo de análisis de un ataque real y las seis respuestas de reflexión.
☐	Ninguna captura muestra una clave privada, frase semilla o URL de RPC con API key.

15 Rúbrica

ESCALA DE 0 A 5,0

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (< 3,0)
Mini-AMM	35 %	15/15 con código propio; salida del paso 4 idéntica; respuestas 1, 2 y 6 correctas.	12 a 14 pruebas; salida con diferencias explicadas.	Menos de 12, o pruebas modificadas.
Oráculo en Sepolia	30 %	6/6, contrato verificado y lectura por el consumidor.	6/6 y desplegado sin verificar.	Sin evidencia en Sepolia.
Hoja de cálculo	20 %	Cuatro ✓ con fórmulas; respuestas 3 y 4 con cálculo.	Tres ✓, o respuestas sin cálculo.	Valores tecleados o menos de tres ✓.
Análisis	15 %	Ataque real bien descrito y ligado a una defensa concreta del laboratorio.	Descripción correcta sin conexión con el laboratorio.	Genérico o con datos sin fuente.

16 Referencias

- Uniswap v2 · *How Uniswap works*. <https://docs.uniswap.org/contracts/v2/concepts/protocol-overview/how-uniswap-works>
- Chainlink · *Price Feed Contract Addresses*. <https://docs.chain.link/data-feeds/price-feeds/addresses>
- Aave v3 · documentación (factor de salud y liquidaciones). <https://aave.com/docs>
- Federal Reserve Bank of Richmond (2022). *Why Stablecoins Fail: An Economist's Post-Mortem on Terra*, Economic Brief 22-24. https://www.richmondfed.org/publications/research/economic_brief/2022/eb_22-24
- U.S. Attorney's Office, SDNY (11-dic-2025). Sentencia de Do Kwon. <https://www.justice.gov/usao-sdny/pr/crypto-enabled-fraudster-sentenced-orchestrating-40-billion-fraud>
- Tablero de ataques a DeFi: <https://rekt.news/leaderboard/> y <https://defillama.com/hacks>